Lecture 02: R and RStudio

EEFE 530

Daniel Brent

Agenda

Today

Why R?

1. Overview
2. Installation
3. RStudio

Basics

R

What is it?

The **R project website**:

> R is a free software environment for statistical computing and graphics. It compiles and runs on a wide variety of UNIX platforms, Windows and MacOS.

What does that mean?

- **R** is a statistical computing language that is flexible for many tasks used in applied econometrics.
- **R** has a ton of online help forums in addition to help files by package developers (*e.g.*, **Stack Overflow**). -more on packages soon.
- **R** is **free** and **open source**.

Why are we using **R**?

1. `R` is **free** and **open source**—saving both you and the university money.
2. Economists are (gradually/quickly??) switching to `R` along with data scientists, and researchers in other fields.
3. More employers favor `R` over proprietary software like `Stata`.
4. `R` is very **flexible and powerful**—adaptable to nearly any task, *e.g.*, 'metrics, spatial data analysis, machine learning, web scraping, data cleaning, website building, teaching.
5. *Related:* `R` imposes **no limitations** on your amount of observations, variables, memory, or processing power and allows multiple objects at once (more on objects soon).
6. With work you'll develop a marketable tool that will help with your research or other career paths.
7. I personally shifted from `Stata` to `R`. Sometimes I go back and forth (not recommended).

The install

Installing `R` is fairly straightforward, but it occasionally involves challenges for older computers. You also have to keep version updates on your mind because some packages are version specific.

Step 1: Download (**r-project.org**) and install `R` for your operating system.

Step 2: Download (**rstudio.com**) and install `RStudio` Desktop for your operating system.

I always just use all the default prompts for the installation.

DataCamp has a nice tutorial on installing `R` and `RStudio` for Windows, Mac, and Linux operating systems.

RStudio

RStudio

- RStudio is an integrated development environment (IDE) for **R**.
- Another way of saying this is that RStudio is the front end and **R** is the back end.
- There are other options, but RStudio is free, widely used, and pretty user-friendly.
- We'll now take some time to explore some of the features of RStudio

RStudio IDE : : **CHEAT SHEET**

Documents and Apps

Open Shiny, R Markdown, knitr, Sweave, LaTeX, .Rd files and more in Source Pane

Check spelling · Render output · Choose output format · Choose output location · Insert code chunk

Jump to previous chunk · Jump to next chunk · Run selected lines · Publish to server · Show file outline

Access markdown guide at **Help > Markdown Quick Reference**

Jump to chunk · Set knitr chunk options · Run this and all previous code chunks · Run this code chunk

RStudio recognizes that files named **app.R**, **server.R**, **ui.R**, and **global.R** belong to a shiny app

Run app · Choose location to view app · Publish to shinyapps.io or server · Manage publish accounts

Write Code

Navigate tabs · Open in new window · Save · Find and replace · Compile as notebook · Run selected code

Cursors of shared users · Re-run previous code · Source with or without Echo · Show file outline

Multiple cursors/column selection with **Alt + mouse drag**.

Code diagnostics that appear in the margin. Hover over diagnostic symbols for details.

Syntax highlighting based on your file's extension

Tab completion to finish function names, file paths, arguments, and more.

Multi-language code snippets to quickly use common blocks of code.

Jump to function in file · Change file type

Working Directory · Press ↑ to see command history · Maximize, minimize panes · Drag pane boundaries

R Support

Import data with wizard · History of past commands to run/copy · Display .RPres slideshows **File > New File > R Presentation**

Load workspace · Save workspace · Delete all saved objects · Search inside environment

Choose environment to display from list of parent environments · Display objects as list or grid

Displays saved objects by type with short description · View in data viewer · View function source code

Create folder · Upload file · Delete file · Rename file · Change directory

Path to displayed directory

A File browser keyed to your working directory. Click on file or directory name to open.

Pro Features

Share Project with Collaborators · Active shared collaborators

Start **new R Session** in current project · Close R Session in project · **Select R Version**

Name of current project

PROJECT SYSTEM

File > New Project

RStudio saves the call history, workspace, and working directory associated with a project. It reloads each when you re-open a project.

RStudio opens plots in a dedicated Plots pane

Navigate recent plots · Open in window · **Export plot** · Delete plot · Delete all plots

GUI Package manager lists every installed package

Install Packages · Update Packages · Create reproducible package library for your project

Click to load package with **library()**. Unclick to detach package with **detach()** · Package version installed · Delete from library

RStudio opens documentation in a dedicated Help pane

Home page of helpful links · Search within help file · Search for help file

Viewer Pane displays HTML content, such as Shiny apps, RMarkdown reports, and interactive visualizations

Stop Shiny app · Publish to shinyapps.io, rpubs, RSConnect, … · Refresh

View(<data>) opens spreadsheet like view of data set

Filter rows by value or value range · Sort by values · Search for value

Debug Mode

Open with **debug()**, **browser()**, or a breakpoint. RStudio will open the debugger mode when it encounters a breakpoint while executing code.

Launch debugger mode from origin of error · Open traceback to examine the functions that R called before the error occurred

Click next to line number to add/remove a breakpoint.

Highlighted line shows where execution has paused

Run commands in environment where execution has paused · Examine variables in executing environment · Select function in traceback to debug · Step through code one line at a time · Step into and out of functions to run · Resume execution · Quit debug mode

Version Control with Git or SVN

Turn on at **Tools > Project Options > Git/SVN**

Stage files: · Show file diff · Commit staged files · Push/Pull to remote · View History

Added · Deleted · Modified · Renamed · Untracked

Open shell to type commands · current branch

Package Writing

File > New Project > New Directory > R Package

Turn project into package.
Enable roxygen documentation with **Tools > Project Options > Build Tools**

Roxygen guide at **Help > Roxygen Quick Reference**

RStudio

Write Code

Navigate tabs

Open in new window

Save

Find and replace

Compile as notebook

Run selected code

Cursors of shared users

Re-run previous code

Source with or without Echo

Show file outline

Multiple cursors/column selection with **Alt + mouse drag**.

Code diagnostics that appear in the margin. Hover over diagnostic symbols for details.

Syntax highlighting based on your file's extension

Tab completion to finish function names, file paths, arguments, and more.

Multi-language code snippets to quickly use common blocks of code.

Jump to function in file

Change file type

Working Directory

Maximize, minimize panes

Press ↑ to see command history

Drag pane boundaries

R Support

Import data with wizard

History of past commands to run/copy

Display .RPres slideshows **File > New File > R Presentation**

Load workspace

Save workspace

Delete all saved objects

Search inside environment

Choose environment to display from list of parent environments

Display objects as list or grid

Displays saved objects by type with short description

View in data viewer

View function source code

Create folder

Upload file

Delete file

Rename file

Change directory

Path to displayed directory

A File browser keyed to your working directory. Click on file or directory name to open.

RStudio

Download: https://rstudio.com/products/rstudio/download/

Cheat Sheets: https://rstudio.com/resources/cheatsheets/

RStudio Cheat Sheet:
https://rstudio.com/resources/cheatsheets/raw/master/rstudio-ide.pdf

R Basics

Basic structure

Okay - back to `R`. There are a few basics to keep in mind with `R`:

1. Everything is an **object**.

```
foo
```

2. Every object has a **name** and **value**.

```
foo <- 2
```

3. You use **functions** on these objects.

```
mean(foo)
```

4. Functions come in **libraries** (**packages**)

```
library(dplyr)
```

5. `R` will try to **help** you.

```
?dplyr
```

6. R has its **quirks**.

Objects

Everything is an object

Define an object

```
1 a <- 2
```

Now `a` is in the **R** environment

```
1 a
```

```
#> [1] 2
```

Objects

We can define and perform functions on multiple objects

```
1 b <- a + 2
2 b
```

```
#> [1] 4
```

```
1 c <- a + b
2 c
```

```
#> [1] 6
```

the `<-` operator assigns an object e.g. (`c <- a + b`) and then just typing object into the console returns the object's value e.g. `c`

Classes

Objects are in a certain class

Let’s define some more objects

```
numbers <- c(2, 4, 6)

numbers.2 <- c(a, b, c)

letters <- c('a','b','c')

factors <- as.factor(letters)

factors.2 <- as.factor(numbers)
```

Classes

What do these classes look like?

```
1 numbers
```

```
#> [1] 2 4 6
```

```
1 letters
```

```
#> [1] "a" "b" "c"
```

```
1 factors.2
```

```
#> [1] 2 4 6
#> Levels: 2 4 6
```

Classes

We can use the `class` function to determine an object's class

```
class(a)
```

```
#> [1] "numeric"
```

```
class(numbers)
```

```
#> [1] "numeric"
```

```
class(letters)
```

```
#> [1] "character"
```

```
class(factors)
```

```
#> [1] "factor"
```

Functions

Most work in **R** will require functions

- **R** has many built in functions
  - → `mean`
  - → `sd`
  - → `max`
  - → `class`
  - → `summary`
- Many more functions are available through packages (more on that soon)

class: clear

```
1 mean(numbers)
```

```
#> [1] 4
```

```
1 mean(numbers.2)
```

```
#> [1] 4
```

```
1 mean(letters)
```

```
#> [1] NA
```

```
1 mean(factors.2)
```

```
#> [1] NA
```

Help and functions

Q: How do we know which arguments a function requires/accepts?

A: `?`

Meaning you can type `?mean` into your **R** console to find the help file associated with the functions/objects named `mean`.

Double bonus: Use `??mean` to perform a fuzzy search for the term `mean` in all of the help files.

Example function: `mean`

Q: How do we know which arguments a function requires/accepts?

A_2: **`RStudio`** will also try to help you.

- Type a name (*e.g.*, `mean`) into the console; **`RStudio`** will show you some info about the function.
- After you type the name and parentheses (*e.g.*, `mean()`), press **`tab`**, and **`RStudio`** will show you a list of arguments for the function.
- Similar functionality in the script section when you hover over or start typing a function.

We can write our own functions

```
1 our_function <- function(arg1, arg2, arg3) {
2   arg1*arg2*arg3
3 }
```

- Our function will show up in the `Environment` pane of RStudio.
- Functions take inputs.

  → In our case `arg1`, `arg2`, and `arg3`
- Then functions perform some operation on those inputs.

class:clear

```
1 our_function <- function(arg1, arg2, arg3) {
2   arg1*arg2*arg3
3 }
4
5 our_function(arg1=2, arg2=4, arg3=6)
```

```
#> [1] 48
```

```
1 our_function(2,4,6)
```

```
#> [1] 48
```

```
1 our_function(a,b,c)
```

```
#> [1] 48
```

```
1 our_function('a','b','c')
```

```
#> Error in arg1 * arg2: non-numeric argument to binary operator
```

We can also save the output of a function to an object

```
our_function <- function(arg1, arg2, arg3) {
  arg1*arg2*arg3
}

output <- our_function(2,4,6)

output
```

```
#> [1] 48
```

Data frames

Data frames are the base spreadsheet style way of storing data in **R**

```
df <- data.frame(x1 = c(a,b,c),
                 x2 = numbers,
                 x3 = letters,
                 x4 = factors,
                 x5 = c(1,2,3))
df
```

```
#>   x1 x2 x3 x4 x5
#> 1  2  2  a  a  1
#> 2  4  4  b  b  2
#> 3  6  6  c  c  3
```

- Now we have a data frame called `df` that has the variables (columns) `x1, x2, x3, x4, x5`
- We can have the variables as existing objects (`x1-x4`)or newly created data (`x5`)

We isolate a specific column of a data frame with $

```
df$x1
```

```
#> [1] 2 4 6
```

```
df$x3
```

```
#> [1] "a" "b" "c"
```

```
df$x4
```

```
#> [1] a b c
#> Levels: a b c
```

We can perform operations/functions of these

```
mean(df$x1)
```

```
#> [1] 4
```

```
class(df$x1)
```

```
#> [1] "numeric"
```

We can add variables to data frames

```
df$x6 <- 3

df
```

```
#>   x1 x2 x3 x4 x5 x6
#> 1  2  2  a  a  1  3
#> 2  4  4  b  b  2  3
#> 3  6  6  c  c  3  3
```

```
df$x7 <- seq(1:3)

df
```

```
#>   x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3
```

We can combine data frames by rows (`rbind`) or by columns (`cbind`)

```
1 rbind(df,df)
```

```
#>   x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3
#> 4  2  2  a  a  1  3  1
#> 5  4  4  b  b  2  3  2
#> 6  6  6  c  c  3  3  3
```

```
1 cbind(df,df)
```

```
#>   x1 x2 x3 x4 x5 x6 x7 x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3  6  6  c  c  3  3  3
```

Data frames

We can create new data frames or write over an existing data frame

```
df
```

```
#>   x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3
```

```
df2 <- cbind(df,df)
df2
```

```
#>   x1 x2 x3 x4 x5 x6 x7 x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3  6  6  c  c  3  3  3
```

We can create new data frames or write over an existing data frame

```
df
```

```
#>   x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3
```

```
df <- rbind(df,df)
df
```

```
#>   x1 x2 x3 x4 x5 x6 x7
#> 1  2  2  a  a  1  3  1
#> 2  4  4  b  b  2  3  2
#> 3  6  6  c  c  3  3  3
#> 4  2  2  a  a  1  3  1
#> 5  4  4  b  b  2  3  2
#> 6  6  6  c  c  3  3  3
```

Packages

Packages

- You can install packages through the function `install.packages()` function in R
  - → You need to write the package name in quotes
    e.g. `install.packages('data.table)`
  - → You only need to **install a package once** (until you update R)
  - → You can also install packages through the package tab in RStudio
- You can load a package using `library()` function
  - → You do not need quotes when loading a package
  - → You need to **load packages every time you open R**

Packages

- Packages are open source and provide access to useful functions and data
- You can get help on a package with the `?` function e.g. `?data.table`
- We'll get familiar with packages with both `data.table` for using data and `ggplot2` for plotting.

Data

Data

- We will know start working with data using the `data.table` package
- Recall our data frame, let's make it a little bigger because that's where `data.table` becomes really helpful

```
df <- rbind(df,df,df,df,df,df,df,df,df,df)
df <- rbind(df,df)
dim(df)
```

```
#> [1] 120   7
```

```
head(df,15)
```

```
#>    x1 x2 x3 x4 x5 x6 x7
#> 1   2  2  a  a  1  3  1
#> 2   4  4  b  b  2  3  2
#> 3   6  6  c  c  3  3  3
#> 4   2  2  a  a  1  3  1
#> 5   4  4  b  b  2  3  2
#> 6   6  6  c  c  3  3  3
#> 7   2  2  a  a  1  3  1
#> 8   4  4  b  b  2  3  2
#> 9   6  6  c  c  3  3  3
#> 10  2  2  a  a  1  3  1
#> 11  4  4  b  b  2  3  2
#> 12  6  6  c  c  3  3  3
#> 13  2  2  a  a  1  3  1
#> 14  4  4  b  b  2  3  2
#> 15  6  6  c  c  3  3  3
```

Data Table

- We can't really see all the data frame
- Now we'll convert to a `data.table` and check it out

```
library(data.table)
dt <- as.data.table(df)
dt
```

```
#>         x1    x2     x3     x4    x5    x6    x7
#>      <num> <num> <char> <fctr> <num> <num> <int>
#>   1:     2     2      a      a     1     3     1
#>   2:     4     4      b      b     2     3     2
#>   3:     6     6      c      c     3     3     3
#>   4:     2     2      a      a     1     3     1
#>   5:     4     4      b      b     2     3     2
#>  ---
#> 116:     4     4      b      b     2     3     2
#> 117:     6     6      c      c     3     3     3
#> 118:     2     2      a      a     1     3     1
#> 119:     4     4      b      b     2     3     2
#> 120:     6     6      c      c     3     3     3
```

Data

1. We can load in `RData` data right into **R** and it has an assigned object

```
load('seattle_housing.RData') ## this object is an RData file and already has a name 'rain.sales'
rain.sales
```

```
#> Key: <pin>
#>                pin tract    bg    zip district       date sale.year yearmonth
#>              <num> <int> <int> <fctr>   <fctr>     <Date>     <num>    <char>
#>      1:    1000009 30504     3  98002   AUBURN 2012-10-04      2012   2012-10
#>      2:    1000040 30504     4          AUBURN 2016-06-17      2016   2016-06
#>      3:    1000042 30504     4          AUBURN 2016-07-07      2016   2016-07
#>      4:    1000044 30504     4  98002   AUBURN 2017-03-10      2017   2017-03
#>      5:    1000061 30504     3  98002   AUBURN 2017-05-10      2017   2017-05
#>     ---
#> 184185: 9904000025   700     4  98125  SEATTLE 2013-08-05      2013   2013-08
#> 184186: 9904000063   700     4  98125  SEATTLE 2016-08-27      2016   2016-08
#> 184187: 9906000030  4800     3  98103  SEATTLE 2013-03-21      2013   2013-03
#> 184188: 9906000035  4800     3  98103  SEATTLE 2011-04-08      2011   2011-04
#> 184189: 9906000090  4800     3  98107  SEATTLE 2010-09-13      2010   2010-09
#>             price  sqft   lot stories  beds baths yearbuilt year.ren condition
#>             <num> <int> <int>   <num> <int> <num>     <int>    <int>     <int>
#>      1:  214316.7  1720 10506     1.0     2  1.50      1969        0         4
#>      2:  407379.6  2344  6002     2.0     4  2.50      2016        0         3
#>      3:  391433.9  2134  6002     2.0     3  2.50      2016        0         3
#>      4:  347719.8  2400  8023     1.0     3  1.75      1948        0         5
#>      5:  355414.4  1870  9500     1.0     3  1.50      1958        0         5
#>     ---
#> 184185:  300594.7  1180  9600     1.5     2  1.00      1928        0         3
#> 184186:  349354.7   810  9600     1.0     3  1.00      1937        0         3
```

Data

2. We can load in other types of data such csv files using the `read.csv` function

```
read.csv('seattle_housing.csv')
```

If we don't assign a the dataset as an object **R** will not store it

```
rain.sales <- read.csv('seattle_housing.csv')
```

Note: You need to be careful about loading in csv or text files - check out the `header` and `sep` options.

Data

We can also directly load it and convert it into a `data.table` object using nested functions

```
rain.sales <- as.data.table(read.csv('seattle_housing.csv'))
```

Data Table Stucture

The basic structure of data tables follows `data[i,j]`

- `data` is the name of the object (dataset)
- `[]` allows you to subset the data
- The `data.table` structure is: `[i,j]`
- `i` designates rows
- `j` designates columns (variables)

`data.table` allows easy sub-setting by rows

```
1 rain.sales[1:1000]
```

```
#>            pin tract    bg    zip district       date sale.year yearmonth
#>          <num> <int> <int> <char>   <char>     <char>     <int>    <char>
#>    1:  1000009 30504     3  98002   AUBURN 2012-10-04      2012   2012-10
#>    2:  1000040 30504     4          AUBURN 2016-06-17      2016   2016-06
#>    3:  1000042 30504     4          AUBURN 2016-07-07      2016   2016-07
#>    4:  1000044 30504     4  98002   AUBURN 2017-03-10      2017   2017-03
#>    5:  1000061 30504     3  98002   AUBURN 2017-05-10      2017   2017-05
#>   ---
#>  996: 88000171 25805     3          RENTON 2015-12-30      2015   2015-12
#>  997: 88000173 25805     3  98055   RENTON 2014-10-15      2014   2014-10
#>  998: 88000174 25805     3  98055   RENTON 2010-08-30      2010   2010-08
#>  999: 88000175 25805     3          RENTON 2015-09-24      2015   2015-09
#> 1000: 88000175 25805     3          RENTON 2018-09-07      2018   2018-09
#>          price  sqft   lot stories  beds baths yearbuilt year.ren condition
#>          <num> <int> <int>   <num> <int> <num>     <int>    <int>     <int>
#>    1: 214316.7  1720 10506       1     2  1.50      1969        0         4
#>    2: 407379.6  2344  6002       2     4  2.50      2016        0         3
#>    3: 391433.9  2134  6002       2     3  2.50      2016        0         3
```

`data.table` allows easy sub-setting by rows

```
1 rain.sales[district=='SEATTLE',]
```

```
#>              pin tract    bg    zip district       date sale.year yearmonth
#>            <num> <int> <int> <char>   <char>     <char>     <int>    <char>
#>     1:    1800066 10900     2  98108  SEATTLE 2013-07-11      2013   2013-07
#>     2:    1800073 10900     2         SEATTLE 2017-10-16      2017   2017-10
#>     3:    1800074 10900     2         SEATTLE 2017-10-13      2017   2017-10
#>     4:    1800075 10402     2  98108  SEATTLE 2010-12-29      2010   2010-12
#>     5:    1800075 10402     2  98108  SEATTLE 2016-03-17      2016   2016-03
#>    ---
#> 56202: 9904000025   700     4  98125  SEATTLE 2013-08-05      2013   2013-08
#> 56203: 9904000063   700     4  98125  SEATTLE 2016-08-27      2016   2016-08
#> 56204: 9906000030  4800     3  98103  SEATTLE 2013-03-21      2013   2013-03
#> 56205: 9906000035  4800     3  98103  SEATTLE 2011-04-08      2011   2011-04
#> 56206: 9906000090  4800     3  98107  SEATTLE 2010-09-13      2010   2010-09
#>            price  sqft   lot stories  beds baths yearbuilt year.ren condition
#>            <num> <int> <int>   <num> <int> <num>     <int>    <int>     <int>
#>     1:  388479.6  1540  6000     1.0     3  1.75      1926        0         4
#>     2:  719749.1  1500  1171     3.0     3  2.00      2017        0         3
#>     3:  640943.1  2390   951     3.0     3  2.00      2017        0         3
```

`data.table` allows easy sub-setting by columns

```
1 rain.sales[,1:6]
```

```
#>                pin tract    bg    zip district       date
#>              <num> <int> <int> <char>   <char>     <char>
#>      1:    1000009 30504     3  98002   AUBURN 2012-10-04
#>      2:    1000040 30504     4          AUBURN 2016-06-17
#>      3:    1000042 30504     4          AUBURN 2016-07-07
#>      4:    1000044 30504     4  98002   AUBURN 2017-03-10
#>      5:    1000061 30504     3  98002   AUBURN 2017-05-10
#>     ---
#> 184185: 9904000025   700     4  98125  SEATTLE 2013-08-05
#> 184186: 9904000063   700     4  98125  SEATTLE 2016-08-27
#> 184187: 9906000030  4800     3  98103  SEATTLE 2013-03-21
#> 184188: 9906000035  4800     3  98103  SEATTLE 2011-04-08
#> 184189: 9906000090  4800     3  98107  SEATTLE 2010-09-13
```

`data.table` allows easy sub-setting by columns

```
rain.sales[,c(1:6,9)]
```

```
#>                pin tract    bg    zip district       date     price
#>              <num> <int> <int> <char>   <char>     <char>     <num>
#>      1:    1000009 30504     3  98002   AUBURN 2012-10-04  214316.7
#>      2:    1000040 30504     4         AUBURN 2016-06-17  407379.6
#>      3:    1000042 30504     4         AUBURN 2016-07-07  391433.9
#>      4:    1000044 30504     4  98002   AUBURN 2017-03-10  347719.8
#>      5:    1000061 30504     3  98002   AUBURN 2017-05-10  355414.4
#>     ---
#> 184185: 9904000025   700     4  98125  SEATTLE 2013-08-05  300594.7
#> 184186: 9904000063   700     4  98125  SEATTLE 2016-08-27  349354.7
#> 184187: 9906000030  4800     3  98103  SEATTLE 2013-03-21 1370406.9
#> 184188: 9906000035  4800     3  98103  SEATTLE 2011-04-08  509918.1
#> 184189: 9906000090  4800     3  98107  SEATTLE 2010-09-13  486352.1
```

`data.table` allows easy sub-setting by columns

```
rain.sales[,list(pin,tract,bg,zip,district,date,price)]
```

```
#>                pin tract    bg    zip district       date     price
#>              <num> <int> <int> <char>   <char>     <char>     <num>
#>      1:    1000009 30504     3  98002   AUBURN 2012-10-04  214316.7
#>      2:    1000040 30504     4          AUBURN 2016-06-17  407379.6
#>      3:    1000042 30504     4          AUBURN 2016-07-07  391433.9
#>      4:    1000044 30504     4  98002   AUBURN 2017-03-10  347719.8
#>      5:    1000061 30504     3  98002   AUBURN 2017-05-10  355414.4
#>     ---
#> 184185: 9904000025   700     4  98125  SEATTLE 2013-08-05  300594.7
#> 184186: 9904000063   700     4  98125  SEATTLE 2016-08-27  349354.7
#> 184187: 9906000030  4800     3  98103  SEATTLE 2013-03-21 1370406.9
#> 184188: 9906000035  4800     3  98103  SEATTLE 2011-04-08  509918.1
#> 184189: 9906000090  4800     3  98107  SEATTLE 2010-09-13  486352.1
```

`data.table` allows easy sub-setting by both rows and columns

```
rain.sales[district=='SEATTLE',list(pin,tract,bg,zip,district,date,price)]
```

```
#>              pin tract    bg    zip district       date     price
#>            <num> <int> <int> <char>   <char>     <char>     <num>
#>     1:    1800066 10900     2  98108  SEATTLE 2013-07-11  388479.6
#>     2:    1800073 10900     2         SEATTLE 2017-10-16  719749.1
#>     3:    1800074 10900     2         SEATTLE 2017-10-13  640943.1
#>     4:    1800075 10402     2  98108  SEATTLE 2010-12-29  382810.4
#>     5:    1800075 10402     2  98108  SEATTLE 2016-03-17  598862.7
#>    ---
#> 56202: 9904000025   700     4  98125  SEATTLE 2013-08-05  300594.7
#> 56203: 9904000063   700     4  98125  SEATTLE 2016-08-27  349354.7
#> 56204: 9906000030  4800     3  98103  SEATTLE 2013-03-21 1370406.9
#> 56205: 9906000035  4800     3  98103  SEATTLE 2011-04-08  509918.1
#> 56206: 9906000090  4800     3  98107  SEATTLE 2010-09-13  486352.1
```

We can also perform functions on different subsets of the data

```
mean(rain.sales[,price])
```

```
#> [1] 648189.2
```

```
mean(rain.sales[district=='SEATTLE',price])
```

```
#> [1] 701690.3
```

```
mean(rain.sales[district=='SEATTLE' & sale.year==2018,price])
```

```
#> [1] 899981.6
```

We can create variable based on functions applied to groups in the data

```
rain.sales[,mean.price := mean(price,na.rm = T)]

## add a variable by group
rain.sales[,mean.district.price := mean(price,na.rm=T), by = district]

rain.sales[,mean.district.year.price := mean(price,na.rm=T), by = c('district','sale.year')]

rain.sales[,list(district,sale.year,price,mean.price,mean.district.price,mean.district.year.price)]
```

```
#>         district sale.year     price mean.price mean.district.price
#>           <char>     <int>     <num>      <num>               <num>
#>      1:   AUBURN      2012  214316.7   648189.2            351953.9
#>      2:   AUBURN      2016  407379.6   648189.2            351953.9
#>      3:   AUBURN      2016  391433.9   648189.2            351953.9
#>      4:   AUBURN      2017  347719.8   648189.2            351953.9
#>      5:   AUBURN      2017  355414.4   648189.2            351953.9
#>     ---
#> 184185:  SEATTLE      2013  300594.7   648189.2            701690.3
#> 184186:  SEATTLE      2016  349354.7   648189.2            701690.3
#> 184187:  SEATTLE      2013 1370406.9   648189.2            701690.3
#> 184188:  SEATTLE      2011  509918.1   648189.2            701690.3
#> 184189:  SEATTLE      2010  486352.1   648189.2            701690.3
#>         mean.district.year.price
#>                            <num>
#>      1:                 274819.3
#>      2:                 371441.2
#>      3:                 371441.2
#>      4:                 405796.5
#>      5:                 405796.5
#>     ---
#> 184185:                 601515.2
#> 184186:                 750155.2
#> 184187:                 601515.2
```

Plotting

Plotting

R has a lot of great graphing tools, but we'll primarily be using `ggplot2`

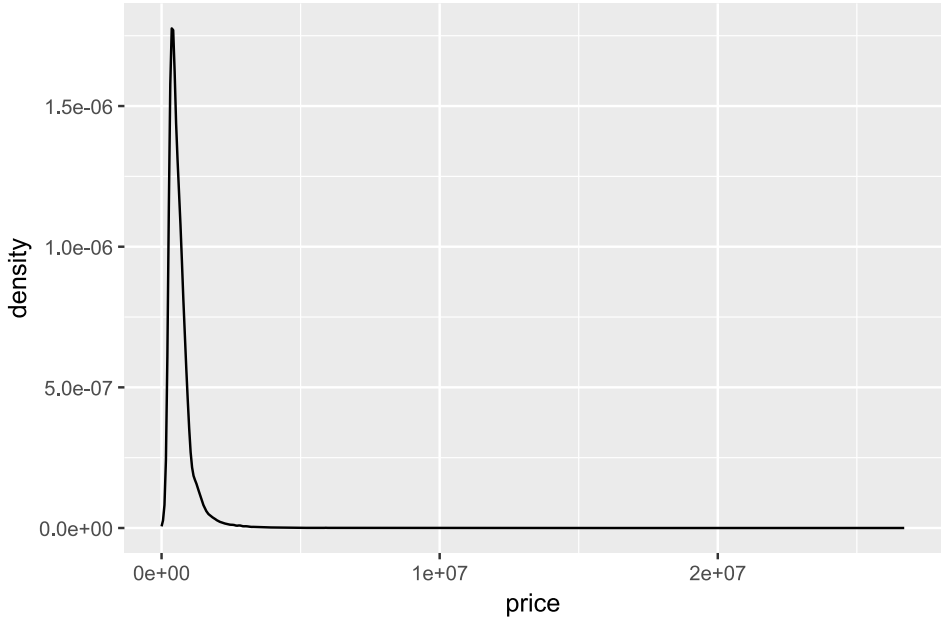
- `ggplot2` has a lot of great functionality and produces great looking graphs

```
1 library(ggplot2)
```

- `ggplot2` will graph objects directly and you can also save `ggplot2` objects and export them as pdf, png, and other file formats

Let's start graphing some densities of housing prices using `ggplot2`

```
1  ggplot(data=rain.sales,aes(x=price)) + geom_density()
```

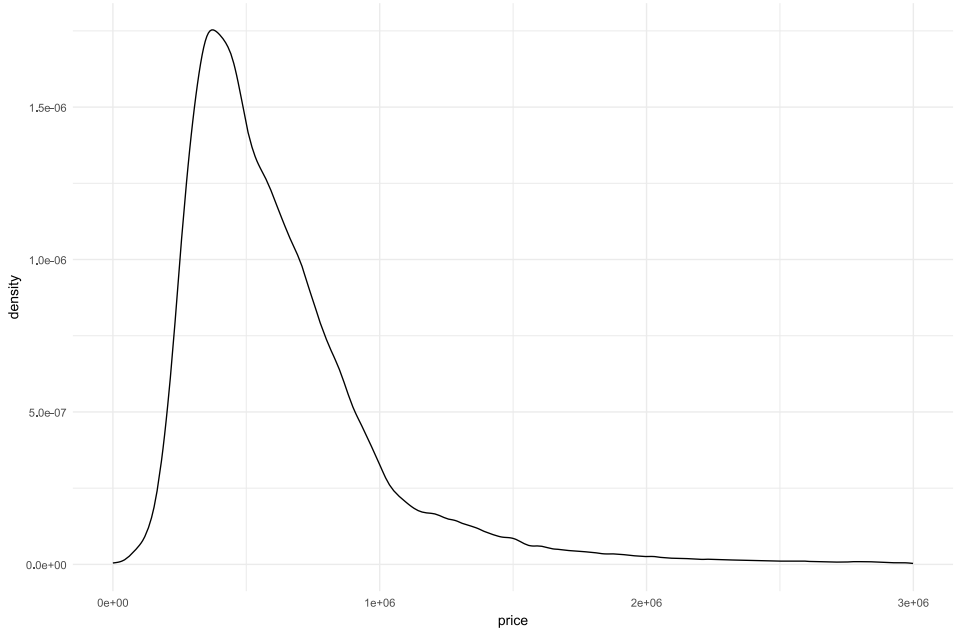


This doesn't look so great so maybe we can improve it a bit.

Plotting

- Let's get rid of far outliers above $3 million
- Let's also change the `ggplot` theme using `theme_minimal()`

```
1 ggplot(data=rain.sales[price<3000000],aes(x=price)) + geom_density() + theme_minimal()
```

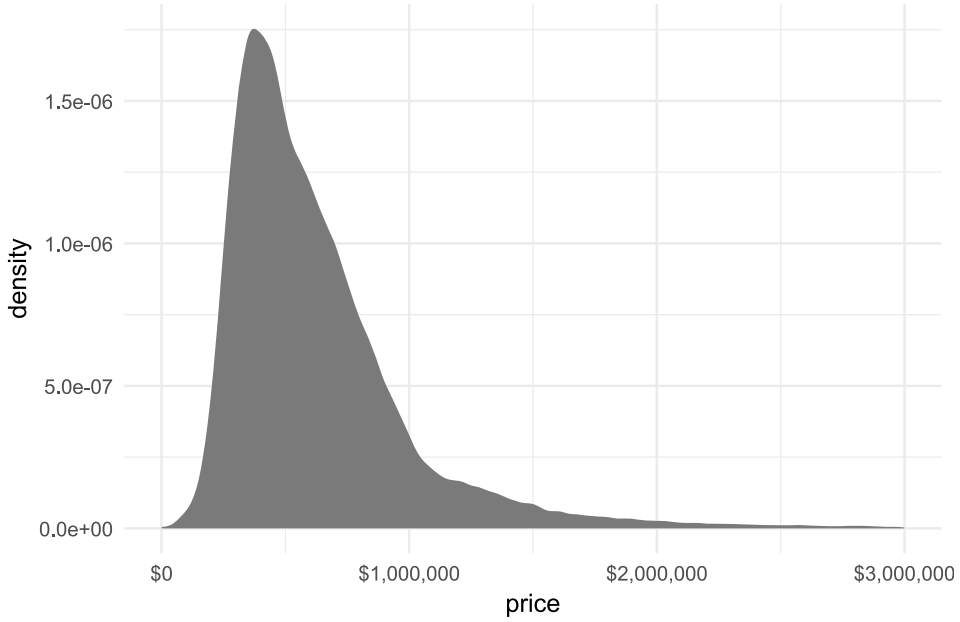


Looks better but we can still improve it.

Plotting

- Now we'll use the `scales` package to add dollars to the x-axis
- We'll also increase the font size and add a bit of color

```
library(scales)
ggplot(data=rain.sales[price<3000000],aes(x=price)) +
  geom_density(fill='dodgerblue',color=NA) +
  scale_x_continuous(labels=scales::dollar) + theme_minimal(base_size = 20)
```



Nice. Now let’s see some additional features.

Plotting

- Now we'll focus on a subset of cities in King County and see the distributions by city by adding `fill=district` into the `aes()` portion of ggplot
- We'll also change the transparency using `alpha`

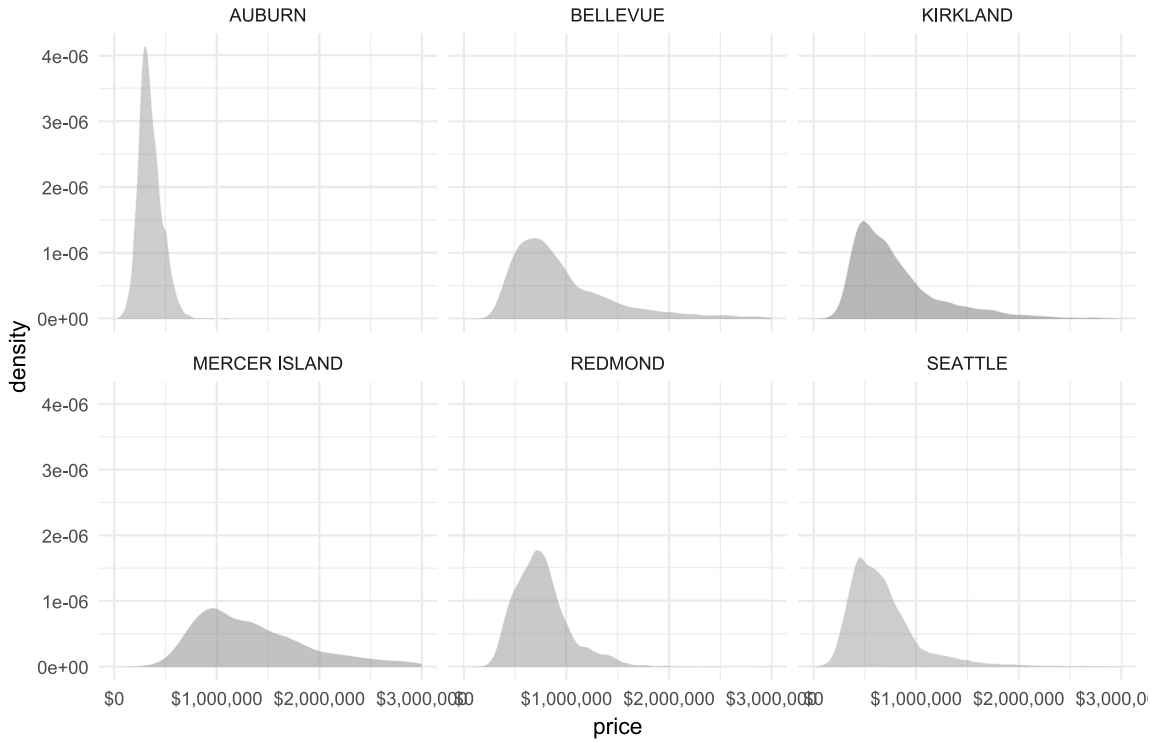
```
ggplot(data = rain.sales[price < 3e+06 & district %in% c("SEATTLE", "BELLEVUE", "AUBURN",
    "BOTHEL", "MERCER ISLAND", "REDMOND", "KIRKLAND")], aes(x = price, fill = district)) +
    geom_density(color = NA, alpha = 0.5) + scale_fill_discrete(name = "City") +
    scale_x_continuous(labels = scales::dollar) + theme_minimal(base_size = 20)
```

Looks good, but kind of hard to read.

Plotting

- Let's use `facet_wrap` to have each city's density in it's own panel

```
ggplot(data=rain.sales[price<3000000 &
                       district %in% c('SEATTLE','BELLEVUE','AUBURN','BOTHEL','MERCER ISLAND','REDMOND','KIRKLAND')],
       aes(x=price, fill = district)) + geom_density(color=NA, alpha=0.5) +
  facet_wrap(~district) + scale_fill_discrete(guide=FALSE) +
  scale_x_continuous(labels=scales::dollar) + theme_minimal(base_size = 15)
```



Great! We can change the angle of the x-axis text to make it fit better, but I'm not going to do that now.

Regression

Regression

Now we'll introduce some basic regression functionality in **R**

- Base **R** uses the `lm()` (linear model) function to run regressions (similar to Stata's `reg`)
- I almost exclusively save the model output, which creates an `lm` object
- Let's start with a hedonic regression on the impact of nice views on housing prices in Seattle
- In our dataset we have a couple variables for mountain and water views

```
rain.sales[,list(sound,lake.wash,lake.sam,rainier,cascades,territorial)]
```

```
#>         sound lake.wash lake.sam rainier cascades territorial
#>         <int>     <int>    <int>   <int>    <int>       <int>
#>      1:     0         0        0       0        0           0
#>      2:     0         0        0       0        0           0
#>      3:     0         0        0       0        0           0
#>      4:     0         0        0       0        0           0
#>      5:     0         0        0       0        0           0
#>     ---
#> 184185:     0         0        0       0        0           0
#> 184186:     0         0        0       0        0           0
#> 184187:     0         0        0       2        0           2
```

Regression

We’re going to collapse that into two dummy variables using the `ifelse` function

```
rain.sales[,water.view := ifelse(sound>0 | lake.wash>0 | lake.sam>0,1,0)]
rain.sales[,mount.view := ifelse(rainier>0 | cascades>0 | territorial>0,1,0)]
```

The actual variables account for the quality of the view, but we’re take a broad measure of any view at all.

There are a couple things to point out.

1. The first argument defines the regression equation. `~` serves as the equal sign in a model equation: the dependent variable is to the left and the independent variables are to the right separated with `+`.
2. We need to tell .monoR what dataset we're using: `data=rain.sales`. This is the second argument in the `lm` function.
3. We will save the output of the regression to `m1`
4. To see the output we use the `summary()` function

```
m1 <- lm(price ~ water.view + mount.view,
         data=rain.sales)
summary(m1)
```

```
#>
#> Call:
#> lm(formula = price ~ water.view + mount.view, data = rain.sales)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1305769  -242150   -91977   129905 25809131
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   596367       1131  527.22   <2e-16 ***
#> water.view    552372       5520  100.06   <2e-16 ***
#> mount.view    157742       4378   36.03   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 458200 on 184186 degrees of freedom
```

Regression

- This is a simple model without any controls so let's add some controls to reduce the omitted variable bias with these variables.
  -**Q:** Could we have omitted variable bias for estimating the capitalization of views?
- We'll control for spatial and temporal effects for census tracts (`tract`) and sale year `sale.year`.

```
m2 <- lm(price ~ water.view + mount.view + tract + sale.year,
         data=rain.sales)
summary(m2)
```

```
#>
#> Call:
#> lm(formula = price ~ water.view + mount.view + tract + sale.year,
#>     data = rain.sales)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1407858  -228259   -85801   117146 25695978
#>
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept) -7.707e+07  8.574e+05  -89.90   <2e-16 ***
#> water.view   5.427e+05  5.377e+03  100.94   <2e-16 ***
#> mount.view   1.563e+05  4.262e+03   36.68   <2e-16 ***
#> tract       -4.697e+00  9.665e-02  -48.60   <2e-16 ***
#> sale.year    3.860e+04  4.256e+02   90.71   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 445800 on 184184 degrees of freedom
```

Regression

```
m2 <- lm(price ~ water.view + mount.view + tract + sale.year,
         data=rain.sales)
summary(m2)
```

```
#>
#> Call:
#> lm(formula = price ~ water.view + mount.view + tract + sale.year,
#>     data = rain.sales)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1407858  -228259   -85801   117146 25695978
#>
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept) -7.707e+07  8.574e+05  -89.90   <2e-16 ***
#> water.view   5.427e+05  5.377e+03  100.94   <2e-16 ***
#> mount.view   1.563e+05  4.262e+03   36.68   <2e-16 ***
#> tract       -4.697e+00  9.665e-02  -48.60   <2e-16 ***
#> sale.year    3.860e+04  4.256e+02   90.71   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 445800 on 184184 degrees of freedom
#> Multiple R-squared:  0.1664, Adjusted R-squared:  0.1663
#> F-statistic:  9189 on 4 and 184184 DF,  p-value: < 2.2e-16
```

Q: Anything funny about this output?

A: `tract` and `sale.year` are treated as continuous variables.

Regression

- The variable classes are really important when running regressions in **R** (or any other package).
- We can convert the `tract` and `sale.year` to fixed effects if we really want tract and year fixed effects with `as.factor()`.

```
m3 <- lm(price ~ water.view + mount.view + as.factor(tract) + as.factor(sale.year),
         data=rain.sales)
summary(m3)
```

```
#>
#> Call:
#> lm(formula = price ~ water.view + mount.view + as.factor(tract) +
#>     as.factor(sale.year), data = rain.sales)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -2185180  -122589   -18813    86176 25750075
#>
#> Coefficients:
#>                           Estimate Std. Error t value Pr(>|t|)
#> (Intercept)               344563.7    17167.7  20.070  < 2e-16 ***
#> water.view                410541.2     4432.9  92.612  < 2e-16 ***
#> mount.view                146327.2     3326.6  43.987  < 2e-16 ***
#> as.factor(tract)200        14932.1    21364.9   0.699 0.484612
#> as.factor(tract)300       -19779.5    23986.3  -0.825 0.409591
#> as.factor(tract)401        40637.9    25162.0   1.615 0.106302
#> as.factor(tract)402        54603.7    23342.9   2.339 0.019326 *
#> as.factor(tract)500       213466.8    23332.9   9.149  < 2e-16 ***
```

Regression

- There are a lot of tracts and we don't really want to estimate each of those fixed effects.
- This is computationally intensive and slows down the estimation.
- We can estimate models with a lot of fixed effects by demeaning the data using the `feols` function from the `fixest` package
- The syntax is the same, except fixed effects go after `|` in the model equation.

```
library(fixest)
m4 <- feols(price ~ water.view + mount.view |
            tract + sale.year,
          data=rain.sales)
summary(m4)
```

```
#> OLS estimation, Dep. Var.: price
#> Observations: 184,189
#> Fixed-effects: tract: 382,  sale.year: 9
#> Standard-errors: IID
#>            Estimate Std. Error t value  Pr(>|t|)
#> water.view 410541.2    4432.93 92.6117 < 2.2e-16 ***
#> mount.view 146327.2    3326.57 43.9874 < 2.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> RMSE: 331,847.7     Adj. R2: 0.537174
#>                   Within R2: 0.109311
```

Regression

- Let's estimate one more model where we add some additional controls.

```
m5 <- feols(price ~ water.view + mount.view +
              yearbuilt + sqft + lot + beds + baths |
              tract + sale.year,
             se='iid',
             # cluster = 'tract',
           data=rain.sales)
summary(m5)
```

```
#> OLS estimation, Dep. Var.: price
#> Observations: 184,189
#> Fixed-effects: tract: 382,  sale.year: 9
#> Standard-errors: IID
#>                 Estimate  Std. Error   t value   Pr(>|t|)
#> water.view 302278.935031 3704.455822  81.59874  < 2.2e-16 ***
#> mount.view  33809.506177 2786.357767  12.13394  < 2.2e-16 ***
#> yearbuilt    -116.636372   28.871258  -4.03988 5.3501e-05 ***
#> sqft          239.025365    1.257960 190.01025  < 2.2e-16 ***
#> lot             0.483125    0.017696  27.30111  < 2.2e-16 ***
#> beds       -46963.952116  966.771041 -48.57815  < 2.2e-16 ***
#> baths       31574.144410 1475.560843  21.39806  < 2.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> RMSE: 274,763.0     Adj. R2: 0.682701
#>                   Within R2: 0.389388
```

Regression

- Now let's correct for potentially correlation within census tracts in the error term with clustered standard errors.

```
m6 <- feols(price ~ water.view + mount.view +
              yearbuilt + sqft + lot + beds + baths |
              tract + sale.year,
             cluster = 'tract',
           data=rain.sales)
summary(m6)
```

```
#> OLS estimation, Dep. Var.: price
#> Observations: 184,189
#> Fixed-effects: tract: 382,  sale.year: 9
#> Standard-errors: Clustered (tract)
#>                 Estimate   Std. Error  t value   Pr(>|t|)
#> water.view 302278.935031 28612.642527 10.56452  < 2.2e-16 ***
#> mount.view  33809.506177  9364.098259  3.61055 3.4639e-04 ***
#> yearbuilt    -116.636372    82.453644 -1.41457 1.5801e-01
#> sqft          239.025365    12.388418 19.29426  < 2.2e-16 ***
#> lot             0.483125     0.073672  6.55781 1.7810e-10 ***
#> beds       -46963.952116  5967.231242 -7.87031 3.7054e-14 ***
#> baths       31574.144410  4204.525008  7.50956 4.2529e-13 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> RMSE: 274,763.0     Adj. R2: 0.682701
#>                   Within R2: 0.389388
```

Regression

- Finally let's create a table of the results from all our regressions `m1-m6`.
- The `etable` package creates nice tables with a lot of different options.
- This only works with `fixest` objects so I re-ran the original three models using `feols`.
- You can export these to text or latex files, or create tables right in **R** (more on that later).

Regression

```
m1 <- feols(price ~ water.view + mount.view,
          data=rain.sales)
m2 <- feols(price ~ water.view + mount.view + tract + sale.year,
          data=rain.sales)
m3 <- feols(price ~ water.view + mount.view + as.factor(tract) + as.factor(sale.year),
          data=rain.sales)
```

Regression

```
library(etable)
tab <- etable(m1,m2,m3,m4,m5,m6,
       drop = 'as.factor', se.below = T)
tab %>%
  kbl() %>%
  kable_classic(font_size=12) %>%
    kable_styling(bootstrap_options = c("striped", "hover", "condensed")) %>%
    scroll_box(width = "800px", height = "500px")
```

	m1	m2	m3	m4	m5	m6
Dependent Var.:	price	price	price	price	price	price
Constant	596,366.9***	-77,074,739.6***	344,563.7***			
	(1,131.1)	(857,369.9)	(17,167.7)			
water.view	552,372.2***	542,702.4***	410,541.2***	410,541.2***	302,278.9***	302,278.9***
	(5,520.5)	(5,376.8)	(4,432.9)	(4,432.9)	(3,704.5)	(28,612.6)
mount.view	157,741.5***	156,349.5***	146,327.2***	146,327.2***	33,809.5***	33,809.5***
	(4,378.0)	(4,262.3)	(3,326.6)	(3,326.6)	(2,786.4)	(9,364.1)
tract		-4.697***				
		(0.0966)				
sale.year		38,604.1***				
		(425.6)				
yearbuilt					-116.6***	-116.6
					(28.87)	(82.45)
sqft					239.0***	239.0***
					(1.258)	(12.39)
lot					0.4831***	0.4831***
					(0.0177)	(0.0737)
beds					-46,964.0***	-46,964.0***
					(966.8)	(5,967.2)
baths					31,574.1***	31,574.1***
					(1,475.6)	(4,204.5)
Fixed-Effects:	----------	----------	----------	----------	----------	----------